```
#include <stdio.h>
#include <stdlib.h>

struct Node {
  int coeff;
  int exp;
  struct Node *next;
};

struct Node *createNode(int coeff, int exp) {
  struct Node *newNode;

  newNode = (struct Node *)malloc(sizeof(struct Node));

  if (newNode == NULL) {
    printf("Memory allocation failed!\n");
    exit(1);
  }

  newNode->coeff = coeff;
  newNode->exp = exp;
  newNode->next = newNode;

  return newNode;
}

/* Insert terms in descending order of exponent */
void insertTerm(struct Node **head, int coeff, int exp) {
  struct Node *current, *prev, *newNode;

  if (coeff == 0)
    return;

  if (*head == NULL) {
    *head = createNode(coeff, exp);
    return;
  }

  current = *head;
  prev = NULL;

  do {
    if (current->exp == exp) {
      current->coeff += coeff;
```

```
      if (current->coeff == 0) {
        if (current->next == current) {
          free(current);
          *head = NULL;
          return;
        }

        if (current == *head) {
          struct Node *tail = *head;

          while (tail->next != *head)
            tail = tail->next;

          *head = current->next;
          tail->next = *head;
        } else {
          prev->next = current->next;
        }

        free(current);
      }
      return;
    }

    if (current->exp < exp)
      break;

    prev = current;
    current = current->next;

  } while (current != *head);

  newNode = createNode(coeff, exp);

  /* Insert at the beginning */
  if (current == *head && exp > (*head)->exp) {
    struct Node *tail = *head;

    while (tail->next != *head)
      tail = tail->next;

    newNode->next = *head;
    tail->next = newNode;
    *head = newNode;
  }
```

```
    /* Insert at the end */
    else if (current == *head) {
        struct Node *tail = *head;

        while (tail->next != *head)
            tail = tail->next;

        tail->next = newNode;
        newNode->next = *head;
    }
    /* Insert in the middle */
    else {
        prev->next = newNode;
        newNode->next = current;
    }
}

/* Create a polynomial */
struct Node *createPolynomial() {
    struct Node *poly = NULL;
    int n, coeff, exp, i;

    printf("Enter number of terms: ");
    scanf("%d", &n);

    if (n < 0) {
        printf("Invalid number of terms.\n");
        return NULL;
    }

    for (i = 0; i < n; i++) {
        printf("Enter coefficient and exponent: ");

        if (scanf("%d%d", &coeff, &exp) != 2 || exp < 0) {
            printf("Invalid term.\n");
            return poly;
        }

        insertTerm(&poly, coeff, exp);
    }

    return poly;
}
```

```
/* Display a polynomial */
void displayPolynomial(struct Node *head) {
   struct Node *temp;
   int first = 1, c;

   if (head == NULL) {
      printf("0");
      return;
   }

   temp = head;

   do {
      c = temp->coeff;

      if (!first)
         printf(c >= 0 ? " + " : " - ");
      else if (c < 0)
         printf("-");

      if (c < 0)
         c = -c;

      if (temp->exp == 0) {
         printf("%d", c);
      } else {
         if (c != 1)
            printf("%d", c);

         printf("x");

         if (temp->exp != 1)
            printf("^%d", temp->exp);
      }

      first = 0;
      temp = temp->next;

   } while (temp != head);
}
```

```
/* Add two polynomials */
struct Node *addPolynomial(struct Node *p1,
                           struct Node *p2) {
  struct Node *result = NULL;
  struct Node *temp;

  if (p1 != NULL) {
    temp = p1;

    do {
      insertTerm(&result, temp->coeff, temp->exp);
      temp = temp->next;
    } while (temp != p1);
  }

  if (p2 != NULL) {
    temp = p2;

    do {
      insertTerm(&result, temp->coeff, temp->exp);
      temp = temp->next;
    } while (temp != p2);
  }

  return result;
}

/* Free allocated memory */
void freePolynomial(struct Node *head) {
  struct Node *current, *next;

  if (head == NULL)
    return;

  current = head->next;

  while (current != head) {
    next = current->next;
    free(current);
    current = next;
  }

  free(head);
}
```

```
int main() {
    struct Node *p1, *p2, *sum;

    printf("\nCreate First Polynomial\n");
    p1 = createPolynomial();

    printf("\nCreate Second Polynomial\n");
    p2 = createPolynomial();

    printf("\nFirst Polynomial: ");
    displayPolynomial(p1);

    printf("\nSecond Polynomial: ");
    displayPolynomial(p2);

    sum = addPolynomial(p1, p2);

    printf("\nSum of Polynomials: ");
    displayPolynomial(sum);

    printf("\n");

    freePolynomial(p1);
    freePolynomial(p2);
    freePolynomial(sum);

    return 0;
}
```